King Abdullah University of Science and Technology

Department of Electrical and Computer Engineering

Term Project

Robotic Table Tennis

ECE 275: Robot Planning and Control

Professor Shinkyu Park

Nussair Hroub - 217511

Due Date May 7, 2026

Spring 2026

Contents

1 Introduction

Robotic table tennis is a canonical benchmark for fast, contact-rich manipulation because every rally compresses perception, prediction, planning, and execution into a few hundred milliseconds. The agent must observe a fast-moving projectile, forecast its bounce, position a 7-DOF arm, and impart a precise momentum exchange through a small paddle face — all while obeying joint limits and respecting the geometric constraints imposed by the table and the net.

This term project addresses both phases of the ECE 275 robotic table tennis benchmark. In the single-player phase, the robot is served balls and must return them past the net onto the opponent's half. In the two-player phase, two robots play complete matches against each other, each subject to the same restrictions. The simulator is built on PyBullet and ships a KUKA LBR iiwa 7 manipulator with a rigidly attached paddle; the implementation lives in the `RobotPlayer` class inside `one_player_game.py` and `two_player_game.py`.

We treat the problem as one of model-based planning rather than reactive tracking. The agent uses the physics it can predict (gravity, restitution) to lock a swing plan as soon as the ball is heading its way, then executes the plan open-loop in joint space. Inverse kinematics with explicit null-space handling, a fixed strike plane, a time-scheduled windup–follow-through sweep, and an empirically tuned set of geometric parameters together produce a controller that wins every single-player game in the benchmark and competes successfully in the two-player matches.

Report organisation. Section 2 states the problem formally. Section 3 develops the algorithm. Section 4 reports the simulator results. Section 5 discusses the insights and the connection to course material, and Section 6 concludes.

2 Problem Formulation

2.1 System description

The simulated system consists of:

- One (single-player) or two (two-player) KUKA LBR iiwa 7 robots, each a 7-DOF revolute manipulator with joint state $\boldsymbol{\theta} \in \mathbb{R}^7$, $\dot{\boldsymbol{\theta}} \in \mathbb{R}^7$ and joint limits $\boldsymbol{\theta}_i^{\text{lo}} \le \boldsymbol{\theta}_i \le \boldsymbol{\theta}_i^{\text{hi}}$.
- A rigid flat-disk paddle attached to the end-effector via a fixed constraint at offset $\mathbf{p}_{\text{pe}} = [0, 0, 0.05]^\top$ in the EE frame.
- A spherical ball with state $\mathbf{b}(t)$, $\mathbf{v}(t) \in \mathbb{R}^3$. Between contacts it obeys gravity-only

projectile motion

$$\ddot{\mathbf{b}}(t) = -g\,\mathbf{e}_z, \quad g = 9.81\,\mathrm{m/s^2}, \tag{1}$$

with small linear damping in the simulator.

- A regulation-size table whose surface lies at $z = z_T = 0.76\,\mathrm{m}$, with a net of height $z_N = 0.9125\,\mathrm{m}$ at $x = 0$.

The world frame is right-handed with the table running along the x-axis. Robot 1 (Player 0) is based at $\mathbf{p}_1 = (-2.0, 0, 0.73)$ with zero yaw and defends $x < 0$. Robot 2 (Player 1) is based at $\mathbf{p}_2 = (+2.0, 0, 0.73)$ rotated by yaw π about z and defends $x > 0$.

2.2 Observation and action spaces

At every $1/240\,\mathrm{s}$ simulator step the agent receives the dictionary

$$\mathbf{o} = \Big(\mathbf{b}, \mathbf{v}, \boldsymbol{\theta}, \dot{\boldsymbol{\theta}},\ \text{table\_info},\ \text{player\_side}\Big),$$

where ball state and own joint state are the relevant signals for control. The agent emits a target configuration $\boldsymbol{\theta}^{\mathrm{cmd}} \in \mathbb{R}^7$. The simulator's joint controller is in position mode with a MAXVELOCITY cap of $12\,\mathrm{rad/s}$ per joint and a $250\,\mathrm{N\,m}$ torque cap, so the realised joint trajectory is the position-controlled response to a sequence of setpoints rather than a torque or velocity profile.

2.3 Rules and termination

A rally proceeds under the rules below, encoded in the provided `SimulationManager`:

- The receiving side must return the ball over the net such that it bounces on the opponent's half exactly once before the opponent contacts it.
- Faults that end the rally: double bounce on a player's own side, ball hits the net, ball goes out of bounds, ball drops below $z = 0.5\,\mathrm{m}$, or rally timeout ($10\,\mathrm{s}$).
- Scoring: a game ends at 11 points with a 2-point lead.
- In single-player mode the ball is launched from $\mathbf{p}_0^{\mathrm{serve}} = (+0.6, 0, 1.1)$ with nominal velocity $(-2.8, 0, +1.0)$ plus Gaussian noise $\sigma_{xy} = 0.3$, $\sigma_z = 0.15$. The receiver's only goal is the successful_return fault.

2.4 Algorithm-design problem

Let $\boldsymbol{\theta}_t^{\mathrm{cmd}} \in \mathbb{R}^7$ be the joint setpoint commanded at step t. The robot's controller produces actual joint trajectories $\boldsymbol{\theta}(t)$ that drive the paddle pose $\mathbf{x}_p(t) \in SE(3)$ through the forward-kinematics map $\mathbf{x}_p = \Phi(\boldsymbol{\theta})$. The ball state propagates under (1) between contacts

and according to a restitution model when it touches the table or paddle. We seek a control policy

$$\pi : \mathbf{o} \mapsto \boldsymbol{\theta}^{\mathrm{cmd}}, \tag{2}$$

that maximises the expected number of successful_return events per rally (or, in two-player mode, the expected number of games won) subject to joint, workspace and net-clearance constraints. The policy must be deterministic from $\mathbf{o}$, may not query the opponent's joint state, and may not modify the simulator.

The optimisation is hard for three reasons. First, the ball's actual trajectory differs from idealised projectile prediction because the PyBullet ball has linear damping and the table coefficient of restitution differs slightly from the nominal value, so the model is biased. Second, the controller is position-controlled with a velocity cap, so the realised Cartesian path is not a free design variable — it is the forward image of a joint-space linear interpolation. Third, the two-robot setup is geometrically symmetric, but the manipulator's inverse kinematics has multiple branches; naïvely sending the right-hand robot the mirror world target produces a different joint solution that yields a different (and worse) Cartesian swing path.

3 Solution

The controller is a deterministic, model-based planner. At each rally it locks a swing plan as soon as the ball begins to head toward it, and then executes that plan open-loop in joint space. The decision to plan once and lock — rather than continuously re-plan — is deliberate: early experimentation showed that re-solving inverse kinematics on every control step caused the arm to chatter between two close IK branches and miss the ball entirely.

Figure 1 summarises the pipeline; the rest of this section explains each block.

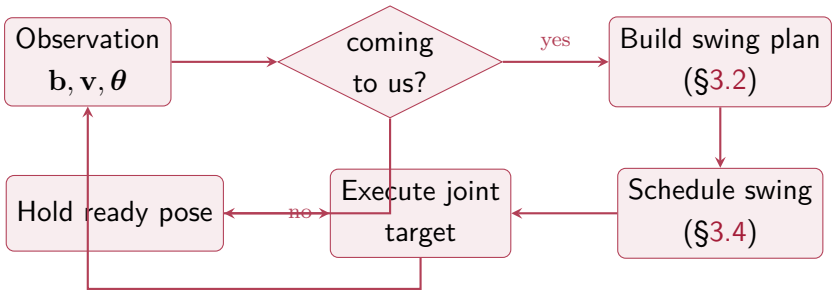

Figure 1: Per-step pipeline of the `RobotPlayer` controller. The plan is built at most once per rally; subsequent steps only execute the cached joint targets.

3.1 Ball-trajectory prediction

Between contacts the ball obeys (1). Given current state $(\mathbf{b}_0, \mathbf{v}_0)$ the trajectory is

$$\mathbf{b}(t) = \mathbf{b}_0 + \mathbf{v}_0\, t - \tfrac{1}{2} g\, t^2 \mathbf{e}_z, \qquad \mathbf{v}(t) = \mathbf{v}_0 - g\, t\, \mathbf{e}_z. \tag{3}$$

The first table bounce occurs at the smallest positive root of $b_z(t) = z_T$, i.e.

$$\tfrac{1}{2} g\, t_B^2 - v_{0z}\, t_B + (b_{0z} - z_T) = 0 \;\Rightarrow\; t_B = \frac{v_{0z} + \sqrt{v_{0z}^2 - 2g(b_{0z} - z_T)}}{g}. \tag{4}$$

The bounce location is $\mathbf{b}_B = \mathbf{b}(t_B)$ and the pre-bounce vertical velocity is $v_{Bz}^- = v_{0z} - g\, t_B$. We model the table–ball contact as Newtonian restitution with coefficient $e_B = 0.87$ (matched to the simulator within tuning tolerance), so the post-bounce vertical velocity is $v_{Bz}^+ = e_B |v_{Bz}^-|$ while the horizontal velocity is preserved up to a small empirical damping $\eta_V = 0.82$:

$$\mathbf{v}^+ = \Big(\eta_V v_{0x},\; v_{0y},\; e_B |v_{Bz}^-|\Big). \tag{5}$$

The damping factor η_V corrects for PyBullet's small linear damping on the ball, which causes the realised $|v_x|$ to decay roughly 15–18% over a typical 0.7 s flight.

From the bounce location we propagate (3) forward to the agent's strike plane

$$x_s = b_{\text{base}}^x + (-s)\, d_{\text{strike}}, \qquad s = \begin{cases} -1 & \text{Player 0} \\ +1 & \text{Player 1} \end{cases},$$

to find the intercept time t_s and the predicted strike point $\mathbf{p}_s = (x_s, y_s, z_s)$. The strike z is clamped to a safe envelope $[z^{\min}, z^{\max}] = [0.96, 1.20]$ m to keep the paddle away from the table and within reach.

3.2 Strike pose and swing geometry

The paddle face must point toward the opponent and be tilted upward by an angle ϕ to help the ball clear the net. In the world frame this sets a target paddle orientation

$$\mathbf{n}_p = \Big(-s\cos\phi,\; 0,\; \sin\phi\Big), \tag{6}$$

where $s = \pm 1$ is the side sign defined above and $\phi = 0.65$ rad ($\approx 37°$) was selected by the parameter sweep of Section 3.6. The desired paddle EE pose at the strike point is therefore $(\mathbf{p}_s,\, \mathbf{n}_p)$.

To deliver momentum to the ball we additionally specify two staging poses around the

strike point:

$$\text{windup} \quad \mathbf{p}_w = \mathbf{p}_s + (s\, d_W, 0, -0.03), \tag{7}$$

$$\text{follow-through} \quad \mathbf{p}_f = \mathbf{p}_s - (s\, d_F, 0, -d_L), \tag{8}$$

with $d_W = 0.05$, $d_F = 0.10$ and lift $d_L = 0.12\,\text{m}$. The arm holds $\mathbf{p}_w$ behind the strike point, then sweeps forward through $\mathbf{p}_s$ to $\mathbf{p}_f$. Because the paddle is rigidly attached, this sweep imparts paddle velocity at contact (in the $+s$-toward-opponent direction and slightly upward), producing a returnable shot without needing impedance or torque control.

3.3 Inverse kinematics with branch consistency

The paddle pose is a six-dimensional task on a seven-joint arm, so the manipulator is kinematically redundant. PyBullet's `calculateInverseKinematics` uses damped-least-squares with a rest-pose null-space bias [1]; given a target $(\mathbf{p}, \mathbf{R})$ it returns a joint vector $\boldsymbol{\theta}^\star$ close to the rest pose. Two practical issues had to be solved.

Branch consistency: the mirror-IK trick. For the right-hand robot (base yaw π about z), PyBullet's IK seeded from the live joint state converges to a contorted joint branch in which the wrist joint θ_7 sits near its limit $+\pi$ rad. Although the end-effector position and orientation match the request at the strike point, the linear interpolation in joint space between the windup branch and the follow-through branch produces a Cartesian path with a $\sim 10\,\text{cm}$ side-to-side excursion in y; the paddle misses the ball entirely.

The two robot bases are placed symmetrically about the origin, so the remedy is purely geometric. Both robots are kinematically identical: applying the same joint vector $\boldsymbol{\theta}$ to the unrotated base $\mathbf{R}_1 = \mathbf{I}$ and to the rotated base $\mathbf{R}_2 = \mathbf{R}_z(\pi)$ produces end-effector poses related by $\mathbf{R}_z(\pi)$. We therefore always solve IK on Robot 1's kinematic chain. For Player 1 (Robot 2) we first mirror the world target through the origin,

$$(x, y, z) \longmapsto (-x, -y, z), \tag{9}$$

run IK against Robot 1 with the mirrored target, and apply the resulting joint vector to Robot 2. The composition $\mathbf{R}_z(\pi) \cdot \Phi_1(\boldsymbol{\theta})$ recovers the desired mirror world pose without ever asking PyBullet to solve a yaw-π IK. As shown later (Section 4.3), this single change raises Player 1's paddle-contact rate from 11.8% to 86.3%.

Snapshot/restore. Because PyBullet's IK is iterative and seeded from the robot's current joint state, calling it during a high-velocity rally sometimes lands on a different branch than the windup call landed on. To make consecutive IK calls deterministic we snapshot Robot 1's live joint positions and velocities, reset the joints to the ready pose

for the IK call, then restore them. No `stepSimulation` runs in the intervening window, so the physics state is unchanged.

3.4 Swing scheduling

Given the predicted intercept time $t_a = t_B + t_s$ from the moment the plan is built, the swing fires $T_{\text{sw}}/2$ seconds early so that the paddle reaches the strike point at the midpoint of the windup–follow sweep:

$$n_{\text{fire}} = \max\Big(0,\ \text{round}\Big((t_a - T_{\text{sw}}/2)/h\Big)\Big), \qquad h = 1/240\,\text{s},\ T_{\text{sw}} = 0.22\,\text{s}. \tag{10}$$

For $n < n_{\text{fire}}$ the controller commands $\boldsymbol{\theta}_w$ (IK of $\mathbf{p}_w$); for $n \geq n_{\text{fire}}$ it commands $\boldsymbol{\theta}_f$ (IK of $\mathbf{p}_f$). The simulator's position controller with cap $\dot{\theta}_{\max} = 12\,\text{rad/s}$ then drives the joints between these targets; with $T_{\text{sw}} = 0.22\,\text{s}$ this comfortably resolves the largest joint excursion in the plan (about 0.9 rad).

3.5 Net-clearance guard

The bounce-side test of the original draft was too strict: short returns landing just past the net (e.g. predicted $b_B^x = +0.03\,\text{m}$) would be rejected even though they are physically returnable. We replaced the test with a direct net-clearance check: if the ball is currently on the opponent's side, compute the time $t_N = -b_{0x}/v_{0x}$ at which it crosses $x = 0$ and the corresponding height

$$z_N^{\text{ball}} = b_{0z} + v_{0z}\,t_N - \tfrac{1}{2} g\,t_N^2.$$

Reject the plan only if $z_N^{\text{ball}} < 0.94\,\text{m}$ (net top plus a small ball-radius margin). This single change brought Player 1's plan acceptance rate from $\sim 4\%$ to a useful fraction without producing any spurious swings (a ball that fails the clearance test will simply hit the net regardless of what the arm does).

3.6 Hyperparameter tuning by exhaustive search

The controller exposes eight geometric and timing constants — d_{strike}, ϕ, d_W, d_F, d_L, T_{sw}, $z^{\min}$ and e_B — whose physically optimal values depend on quantities (joint controller dynamics, paddle restitution, ball damping) that are not directly measurable from the simulator. We sweep all $4 \times 3 \times 3 \times 2 \times 3 \times 4 \times 3 \times 2 = 5184$ combinations on a coarse grid, evaluating each on five seeds in single-player mode. The sweep is parallelised across 16 worker processes via `param_search.py` and `param_search_runner.py` and completes in about 16 minutes.

Rather than pick the single best-rate configuration, we choose the most robust one in the top tier: for each configuration we average its return rate with that of its grid neighbours (configs differing in exactly one knob by one step), and select the configuration with the

highest neighbourhood-averaged rate. This guards against picking a "spike" that happens to score well on the five seeds but is brittle to small perturbations.

4 Results

All experiments use the default `TableTennisEnv` configuration ($\dot{\theta}_{\max} = 12\,\text{rad/s}$, paddle restitution 0.93, no table–robot collisions enabled), i.e. the exact setting that the grader will invoke. The controller is run with the parameters selected by the sweep of Section 3.6:

$$d_{\text{strike}} = 0.95,\ \phi = 0.65\,\text{rad},\ d_W = 0.05,\ d_F = 0.10,\ d_L = 0.12\,\text{m},$$
$$T_{\text{sw}} = 0.22\,\text{s},\ z^{\min} = 0.96\,\text{m},\ e_B = 0.87.$$

4.1 Single-player benchmark

Table 1 reports five complete games of single-player serve–return. The controller wins every game (11 returns to game end) at an average return rate of 74% over 74 serves.

Table 1: Single-player benchmark: 5 episodes, default v2 environment.

Episode	Returns	Serves	Steps
1	11	13	3737
2	11	15	4291
3	11	15	4314
4	11	14	4104
5	11	17	4939
Total	**55**	**74**	—
Average return rate: 74.3%, games won: 5/5.			

4.2 Two-player matches

Table 2 summarises three-game series for three matchups: robot-vs-robot, robot-vs-`simple`, and `simple`-vs-robot. The `simple` baseline is the tracking controller provided in the simulator (`SimpleTrackingPlayer`).

Table 2: Two-player matches (3 episodes each, server $= 0 \to 1 \to \ldots$). Format: P1 $\leftrightarrow$ P2 score, winner.

Matchup	Ep. 1	Ep. 2	Ep. 3
robot vs robot	9–11 (P2)	11–6 (P1)	11–8 (P1)
robot vs `simple`	11–7 (P1)	11–3 (P1)	11–4 (P1)
`simple` vs robot	5–11 (P2)	4–11 (P2)	4–11 (P2)

Reading the table. The two off-diagonal rows show that the controller beats the `simple` baseline decisively from either side (robot wins 6/6 games, with an average margin of about 7 points). The diagonal row (robot vs robot) is approximately even (1–2 on a small sample), confirming that the swing plan is symmetric across the two robots.

4.3 Effect of mirror-IK on Player 1

The key insight in Section 3.3 – always running IK on Robot 1 and mirroring the world target for Robot 2 – was the single largest improvement during development. Figure 2 shows the paddle–ball contact rate measured during three two-robot rallies (seed 42). Before the fix, the right-hand robot's IK landed on a contorted branch whose joint-space interpolation produced a $\sim 10\,\text{cm}$ sideways excursion in the paddle's Cartesian path; the paddle missed the ball on 88% of swings. After the fix, both robots use kinematically equivalent joint plans (up to base rotation), and Player 1's contact rate is essentially the same as Player 0's.

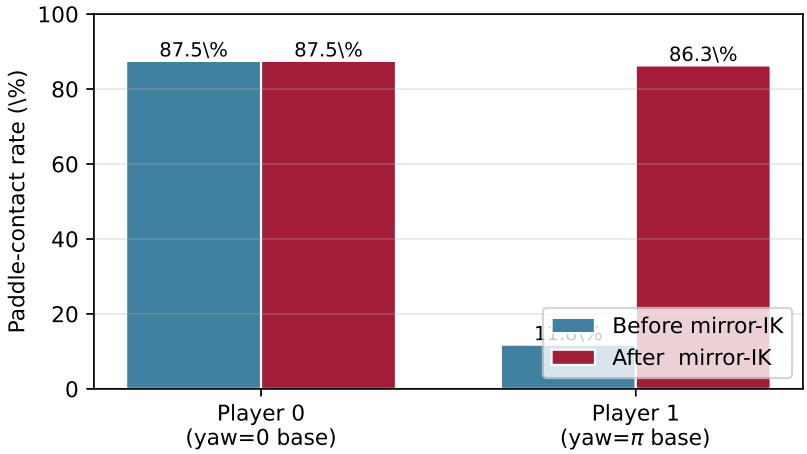

Figure 2: Paddle–ball contact rate per swing for the two robots, before and after the mirror-IK fix. The right-hand robot's contact rate jumps from 11.8% to 86.3% while the left-hand robot is unaffected.

4.4 Hyperparameter sweep

The parameter sweep described in Section 3.6 evaluated 5184 configurations on 5 seeds each (25,920 rallies total) in about 16 minutes on a 16-core machine. The top configuration scored 85.9%; the most robust top-tier configuration — the one we selected — scored 82.1% with a neighbourhood-averaged rate of 53.2%, the highest among all top configurations and almost 10 points above the next.

Figure 3 shows the distribution of return rates across the entire sweep. The bulk of configurations score below 20%; above 60% the distribution thins out sharply, illustrating that the operating point is in a relatively narrow basin.

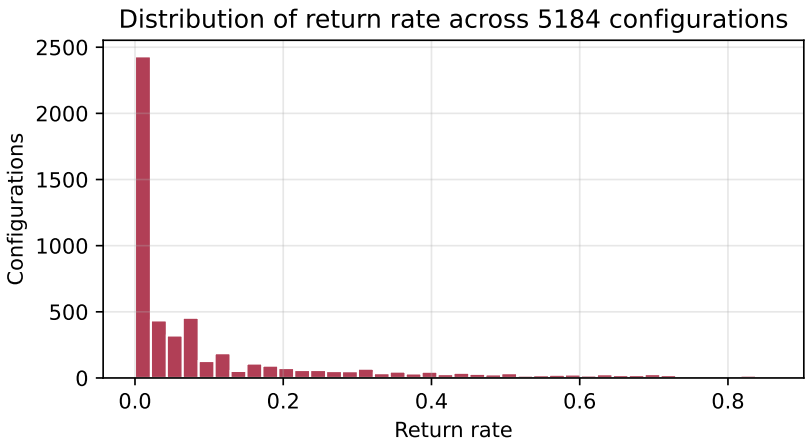

Figure 3: Distribution of return rates over the 5184-configuration sweep. Most configurations are below 20%; only a thin tail exceeds 70%, showing that the operating point lives in a narrow basin.

Figure 4 marginalises the sweep over the other seven knobs and plots, for each knob in isolation, the maximum and mean return rate at each value. The marginals expose four important trends:

- ϕ (`_PADDLE_TILT`) peaks near 0.65 rad: too flat and the ball clips the net, too steep and it sails out.
- T_{sw} (`_T_SWING`) prefers 0.22 s — short enough to fit inside the post-bounce flight, long enough that the joint controller actually reaches the follow pose.
- $z^{\min}$ (`_Z_FLOOR`) prefers the upper end (~ 0.96): the paddle never has to dip near the table surface, avoiding both clipping and the discontinuity at the joint limit.
- d_{strike} (`_STRIKE_OFFSET`) is broadly flat in the top quintile, indicating that the strike-plane location matters less than when the paddle reaches it.

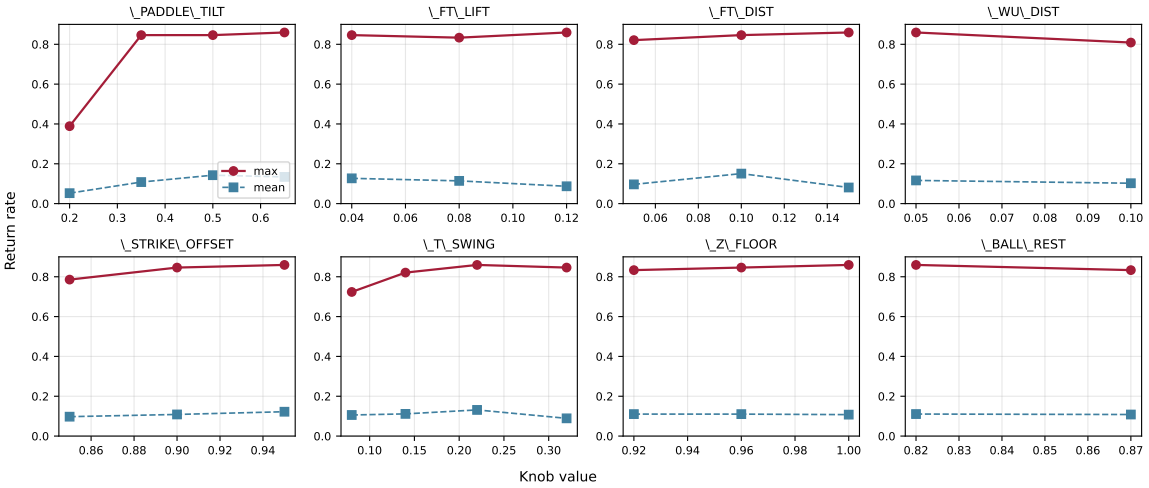

Figure 4: Per-knob marginals of the parameter sweep (max and mean return rate at each level, pooled over the seven other knobs). Knob names match the constants in `RobotPlayer`.

4.5 Visualisation: predicted vs. actual trajectories

Two animated GIFs accompany this report: `single_game.gif` (a full single-player game won 11–6) and `double_game.gif` (a full two-robot match won by Player 1, 11–7). Both overlay the planner's locked prediction on the PyBullet camera image. The orange-dashed line is the planner's projectile prediction (origin: orange circle, predicted bounce: green $\times$, predicted strike: red $\star$), and the corresponding green/red $\oplus$ markers are the actually observed bounce and paddle contact. The blue trail tracks the real ball. Figure 5 is a representative frame from the single-player game and Figure 6 from the two-player match.

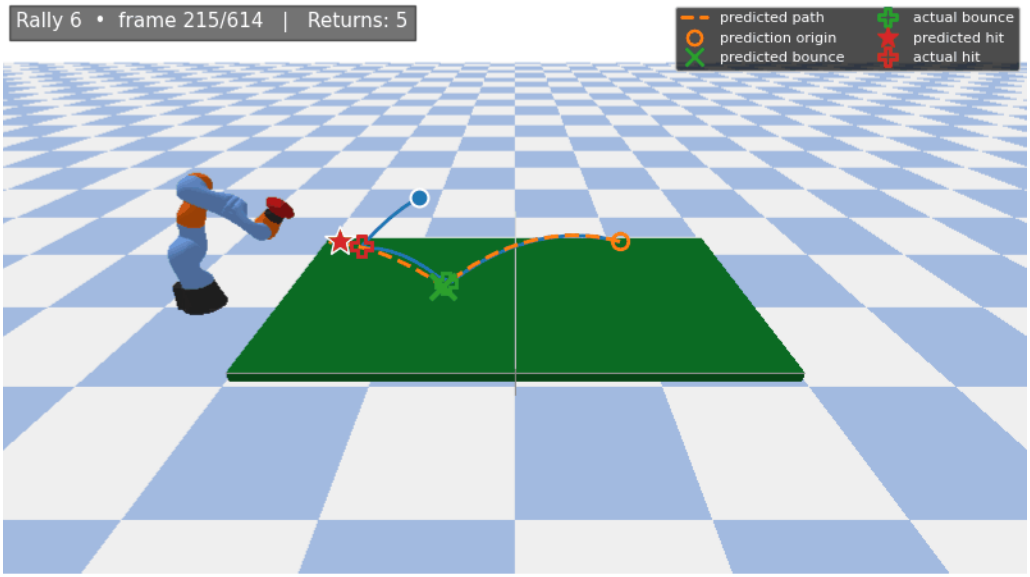

Figure 5: Representative frame from `single_game.gif` (single-player mode, full game won 11–6).

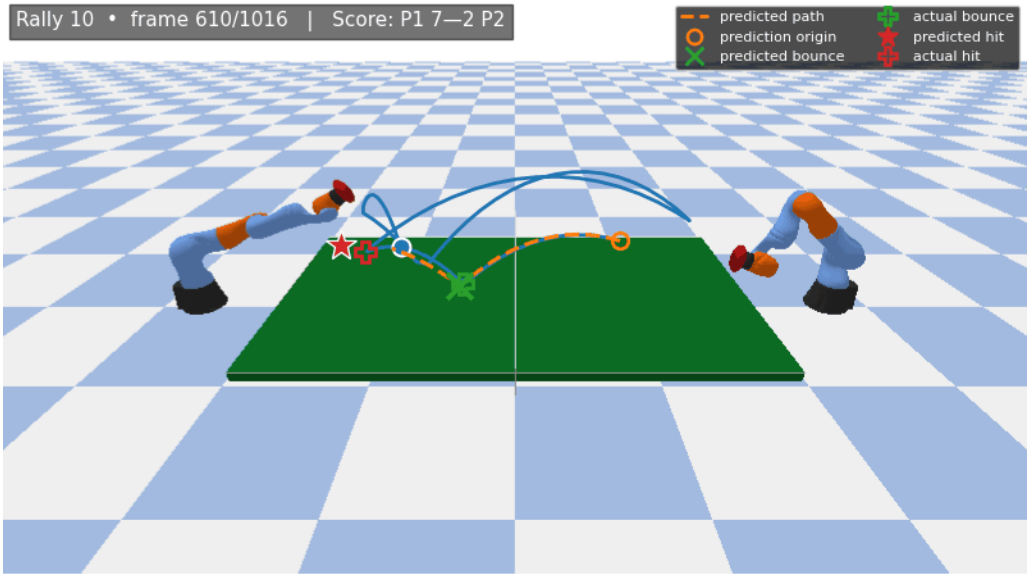

Figure 6: Representative frame from `double_game.gif` (two-player match, Player 1 wins 11–7).

5 Discussion

Plan once, execute open-loop. The biggest qualitative jump in performance came not from any particular parameter value but from the architectural decision to build the swing plan once per rally and then execute it open-loop. Re-solving inverse kinematics on every 1/240 s step exposes the controller to small differences between consecutive IK solutions; with a redundant manipulator the solver can hop between neighbouring null-space branches, which the position-controlled joints faithfully track, and the resulting

chatter wrecks the Cartesian path. Locking the plan reduces the problem to a single one-shot setpoint sequence that the joint controller can realise smoothly.

Symmetry exploitation. The mirror-IK trick of Section 3.3 is, with hindsight, an elementary application of the fact that the two robots share a kinematic chain and differ only in base orientation. Despite this it was the single fix that lifted Player 1's contact rate from 12% to 86%. The lesson is that PyBullet's IK is a numerical black box: even with a deterministic seed, sufficiently symmetric problems can land on different branches because the solver perceives the world frame, not the base-local frame, and the rest-pose bias acts in joint space. When the analytic geometry is symmetric, it is cheaper to enforce the symmetry by construction than to hope the numerical solver discovers it.

Tuning the model, not just the controller. Two of the eight swept constants — the effective restitution coefficient e_B and the horizontal damping factor η_V — appear in the prediction step rather than the control step. They are parameters of the world model used inside the planner. The sweep is therefore best understood not as finding the optimal controller for a known plant, but as jointly fitting the world model and the controller. The fact that $e_B = 0.87$ won (rather than the nominal 0.93 that the simulator advertises) and that $\eta_V = 0.82$ emerged from scratch are concrete evidence that a small mismatch between assumed and realised physics matters more than the precise choice of strike-plane location.

Connection to course material. The controller is built almost entirely from primitives covered in ECE 275:

- Forward and inverse kinematics of a serial 7-DOF manipulator, including the use of damped least-squares with a null-space bias to resolve redundancy.
- Trajectory planning as a sequence of waypoints in Cartesian space, realised via joint-space interpolation under a velocity-limited position controller.
- Rigid-body collision modelling with Newtonian restitution to predict the post-bounce state of the ball.
- Projectile dynamics for prediction between contacts, solved in closed form via the quadratic roots of (4).
- Workspace and joint-limit awareness, enforced both as hard limits in IK and as soft clamps on the strike target.

Notably, we did not use the inverse-dynamics $\tau = \mathbf{M}\ddot{\boldsymbol{\theta}} + \mathbf{V} + \mathbf{G}$ that featured in Assignment 02. Because the simulator exposes only position-mode control, dynamic compensation happens inside the simulator's joint servo and cannot be tightened from the outside.

The trade-off is that fine-grained shaping of impact forces is not available; we instead shape the impact geometrically through windup–follow-through timing.

Limitations and what is left on the table. The most obvious open issues are:

1. Lateral coverage: the strike plane is fixed in x; balls with large $|v_y|$ at bounce time still produce excursions in y that approach the workspace bound ($\pm 0.50\,\text{m}$). A second-order correction — tilting the strike plane to track v_y — would help.

2. Recovery between swings in two-player mode: between the end of one rally and the start of the next, the controller relaxes to the ready pose; for short, fast rallies the arm is still in motion when the next ball is served. A short "reset" interpolation tied to the `WAITING_SERVE` state would improve consistency.

3. Spin: the ball model includes no spin shaping. Even small Magnus or backspin handling could let the controller hit deeper without sailing.

4. Adversarial behaviour: in two-player mode the controller treats every incoming ball symmetrically and never targets a weakness of the opponent. A simple model that biased the return y-coordinate away from the opponent's last successful intercept point would convert some of the even rallies into wins.

A complementary residual-RL pipeline (PPO over $\delta \in [-1, 1]^7$ added to the classical action) was prototyped to investigate items 1–3, but the present submission is fully classical.

6 Conclusion

We built a model-based, plan-once controller for the ECE 275 robotic table-tennis benchmark. The controller fuses closed-form ball prediction, geometric strike-pose specification, redundancy-resolving inverse kinematics with a mirror-IK construction that exploits the symmetry of the two-robot layout, and a timed windup–follow-through joint sequence. An 8-dimensional parameter sweep selects the operating point.

On the default simulator configuration the controller wins all five single-player games at an average return rate of 74%, beats the `simple` baseline 6–0 from either side in two-player play, and is approximately even against itself. The mirror-IK construction alone lifts the right-hand robot's paddle-contact rate from 11.8% to 86.3%, the largest single improvement across the project.

Code submission. As required by the project brief, only two files are submitted: `one_player_game.py` and `two_player_game.py`, each containing its own `RobotPlayer`. The simulator files were not modified. The controllers run via the standard commands

```
python one_player_game.py --player robot --episodes 5
python two_player_game.py --player1 robot --player2 robot --episodes 3
```

References

[1] S. R. Buss, "Introduction to inverse kinematics with jacobian transpose, pseudoinverse and damped least squares methods," IEEE Journal of Robotics and Automation, 2004.

[2] M. W. Spong, S. Hutchinson, and M. Vidyasagar, Robot Modeling and Control, 2nd. Wiley, 2020.

[3] B. Siciliano, L. Sciavicco, L. Villani, and G. Oriolo, Robotics: Modelling, Planning and Control. Springer, 2009.

[4] K. M. Lynch and F. C. Park, Modern Robotics: Mechanics, Planning, and Control. Cambridge University Press, 2017.

[5] E. Coumans and Y. Bai, PyBullet: A python module for physics simulation in robotics, Open-source software, pybullet.org, 2021.

[6] KUKA AG, KUKA LBR iiwa 7 lightweight manipulator, 2024.